Styling React Components

A clear guide to styling in functional components

React gives you **several ways to style your components**. Each method has its own use-case, pros and cons. This guide walks you through the four most common approaches used in **functional components**.

1. Global CSS — Importing a CSS File

The simplest way to add styles is to **create a plain .css file** and import it into your component (or into *index.js / main.jsx*). The styles become available across the entire application.

How it works:

Create a CSS file → styles.css

```
.header { background-color: #4A90D9; color: white; padding: 16px; } .button { border-radius: 6px; padding: 8px 16px; }
```

Import it in your component:

```
import './styles.css'; function App() { return Hello React; }
```

■ Advantages

- Easy to set up — no extra packages needed.
- Great for base/reset styles (e.g. body, font-family).
- Reuse styles across many components easily.
- CSS is familiar to all web developers.

■ Disadvantages

- Styles are global — any component can accidentally override them.
- Class name clashes become a problem in large projects.
- Hard to track which style belongs to which component.
- No automatic scoping or isolation.

2. Inline Styles

You can pass a **JavaScript object** directly to the **style** prop of any JSX element. CSS property names are written in *camelCase* instead of kebab-case.

Example:

```
function Button({ label }) { const btnStyle = { backgroundColor: '#4A90D9', color: 'white', padding: '8px 16px', borderRadius: '6px', border: 'none', cursor: 'pointer', }; return {label}; }
```

■ Advantages
• Styles are scoped to exactly one element — no leaking.
• Can use JS variables and expressions directly in styles.
• Great for quick, one-off dynamic values.
• No class name collisions — ever.

■ Disadvantages
• No support for pseudo-classes like :hover or :focus.
• Cannot use media queries inline.
• Mixes logic and presentation — harder to read.
• Repetitive if many elements share the same style.

3. Dynamic & Conditional Styling with CSS Classes

You can **change which CSS class is applied** based on component state or props. This keeps your CSS in .css files while making the UI react to user actions.

Approach A — Ternary operator:

```
/* styles.css */ .btn { padding: 8px 16px; border-radius: 6px; } .btn-active { background-color: #4A90D9; color: white; } .btn-idle { background-color: #E2E8F0; color: #333; }
import { useState } from 'react'; import './styles.css'; function Toggle() { const [active, setActive] = useState(false); return ( setActive(prev => !prev)} > {active ? 'ON' : 'OFF'} ); }
```

Approach B — Building className dynamically:

```
const classes = ['btn']; if (isActive) classes.push('btn-active'); if (isDisabled) classes.push('btn-disabled'); Click me
```

■ Tip: For complex class combinations, the popular **clsx** or **classnames** package makes this even cleaner.

4. Scoping CSS with CSS Modules

CSS Modules are CSS files where every class name is automatically made unique by the build tool (e.g. Vite, Create React App). This means your styles are **scoped to one component** and will never accidentally affect another component — even if both use the same class name.

Step 1 — Name your file with .module.css:

```
/* Button.module.css */ .button { background-color: #4A90D9; color: white; padding: 8px 20px; border-radius: 6px; border: none; cursor: pointer; } .button:hover { background-color: #2563EB; }
```

Step 2 — Import as an object and use it:

```
import styles from './Button.module.css'; function Button({ label }) { return {label}; }
```

Under the hood, the class name gets transformed to something like **button_a1b2c3** — unique and collision-free.

■ Advantages

- Automatic scoping — zero risk of style leaking.
- Supports all CSS features: hover, media queries, animations.
- No need for long or prefixed class names.
- Works out-of-the-box with Vite and Create React App.

■ Disadvantages

- Slightly more import boilerplate than a plain CSS file.
- Dynamic class names need template literals or clsx.
- Not great for truly global styles (use a separate global CSS for that).
- Less familiar to developers new to CSS Modules.

Quick Comparison

Method	Scoped?	Dynamic?	Best For
Global CSS	No	No	Base / reset styles
Inline Style	Yes	Yes	Quick one-off dynamic values
CSS + Classes	No	Yes	State-driven class toggling
CSS Modules	Yes ■	Yes	Component-scoped reusable styles

■ Code Examples

Explore hands-on code examples for all the styling methods covered in this guide:

https://github.com/academind/react-complete-guide-course-resources/tree/main/code/07 Styling

The repository contains working React projects demonstrating Global CSS, Inline Styles, Dynamic Class-based Styling, and CSS Modules — all inside functional components.
